

Práctica 1

Catálogo de una tienda en línea

Vistas con Blade y Tailwind CSS v4 sobre datos estáticos

Laravel 13 · PHP 8.4 · Tailwind v4 · Sin base de datos · Individual · 100 pts

01 Objetivo y competencias

Al finalizar esta práctica, usted será capaz de construir la capa de presentación (las vistas) de una aplicación web en Laravel 13, consumiendo datos estáticos que el controlador entrega a la vista y maquetando la interfaz con Blade y Tailwind CSS v4.

Competencias específicas:

- Comprender y aplicar el flujo ruta, controlador, vista en Laravel.
- Recibir en una vista Blade los datos que envía el controlador y recorrerlos con directivas (`@forelse`, `@foreach`, `@if`).
- Maquetar una interfaz responsiva mobile-first con utilidades de Tailwind CSS v4 (Flexbox, Grid, estados hover/focus).
- Estructurar vistas reutilizables mediante un layout base y componentes Blade.
- Versionar el trabajo con commits atómicos y documentarlo en un repositorio de GitHub.

02 Descripción del proyecto

Construirá el catálogo de una tienda en línea ficticia llamada **ByteStore**. El catálogo de productos vive en un arreglo dentro del controlador; su trabajo se concentra en las vistas.

Lo que ya se le entrega resuelto en esta guía:

- Las rutas del proyecto (sección 04).
- El controlador con los datos estáticos (sección 05).
- El puente de datos hacia la vista, es decir, cómo la vista recibe las variables (sección 06).

Lo que usted debe desarrollar (el trabajo independiente):

- Todas las vistas Blade + Tailwind: el layout base, el catálogo, al menos un componente y, opcionalmente, la página de detalle.

FLUJO

GET / → TiendaController@index → resources/views/tienda/index.blade.php

03 Preparación del entorno

Trabaje sobre un proyecto Laravel 13 nuevo. Levante Vite para que Tailwind compile y sirva la aplicación.

```
terminal
# Crear el proyecto (si aún no lo tiene)
laravel new tienda-app
cd tienda-app

# Instalar dependencias de frontend y levantar Vite (compila Tailwind)
npm install
npm run dev

# En otra terminal, levantar el servidor de Laravel
php artisan serve
```

Copiar

NOTA

En Laravel 12 y superiores, Tailwind CSS v4 ya viene integrado por defecto (configuración CSS-first, sin `tailwind.config.js`). Solo debe verificar que `resources/css/app.css` contenga la importación de Tailwind:

resources/css/app.css

Copiar

```
@import "tailwindcss";

/* Tailwind ya escanea sus vistas Blade automáticamente.
   Escriba clases de utilidad directamente en los archivos .blade.php */
```

04 Rutas del proyecto

Registre las siguientes rutas en `routes/web.php`. Ya están escritas: solo debe copiarlas.

routes/web.php

Copiar

```
<?php

use App\Http\Controllers\TiendaController;
use Illuminate\Support\Facades\Route;

// Catálogo (página principal)
Route::get('/', [TiendaController::class, 'index'])->name('tienda.index');

// Detalle de un producto (BONUS – opcional)
Route::get('/producto/{id}', [TiendaController::class, 'show'])->name('tienda.show');
```

MÉTODO	URI	NOMBRE	ACCIÓN
GET	/	tienda.index	TiendaController@index
GET	/producto/{id}	tienda.show	TiendaController@show (bonus)

05 Controlador con datos estáticos

Genere el controlador con Artisan:

terminal

Copiar

```
php artisan make:controller TiendaController
```

El controlador define los datos en métodos privados (una **única fuente**) y los envía a la vista con el helper `view()`. Cópielo tal cual:

app/Http/Controllers/TiendaController.php

Copiar

```
<?php

namespace App\Http\Controllers;

class TiendaController extends Controller
{
    // Catálogo: envía la tienda, las categorías y los productos a la vista.
```

```

public function index()
{
    return view('tienda.index', [
        'tienda'      => $this->tienda(),
        'categorias' => $this->categorias(),
        'productos'  => $this->productos(),
    ]);
}

// Detalle de un producto (BONUS).
public function show(int $id)
{
    $producto = collect($this->productos())->firstWhere('id', $id);

    abort_if(is_null($producto), 404);

    return view('tienda.show', [
        'tienda'      => $this->tienda(),
        'producto'    => $producto,
    ]);
}

// ---- Datos estáticos (única fuente de verdad) -----
private function tienda(): array
{
    return ['nombre' => 'ByteStore', 'lema' => 'Tecnología que rinde'];
}

private function categorias(): array
{
    return ['Audio', 'Cómputo', 'Accesorios', 'Wearables'];
}

private function productos(): array
{
    return [
        ['id' => 1, 'nombre' => 'Auriculares Aura X200', 'categoria' => 'Audio',
        'precio' => 249.90, 'stock' => 8, 'destacado' => true,
        'imagen' => 'https://picsum.photos/seed/aura200/600/400',
        'descripcion' => 'Cancelación activa de ruido y 30 h de batería.'],

        ['id' => 2, 'nombre' => 'Teclado Mecánico Nova K87', 'categoria' => 'Cómputo',
        'precio' => 320.00, 'stock' => 5, 'destacado' => false,
        'imagen' => 'https://picsum.photos/seed/novak87/600/400',
        'descripcion' => 'Switches táctiles e iluminación RGB por tecla.'],

        ['id' => 3, 'nombre' => 'Mouse Ergonómico Glide M6', 'categoria' => 'Accesorios',
        'precio' => 95.50, 'stock' => 0, 'destacado' => false,
        'imagen' => 'https://picsum.photos/seed/glidem6/600/400',
        'descripcion' => 'Diseño vertical que reduce la fatiga de muñeca.'],

        ['id' => 4, 'nombre' => 'Smartwatch Pulse S3', 'categoria' => 'Wearables',
        'precio' => 540.00, 'stock' => 3, 'destacado' => true,
        'imagen' => 'https://picsum.photos/seed/pulses3/600/400',
        'descripcion' => 'Monitor de ritmo cardíaco y GPS integrado.'],

        ['id' => 5, 'nombre' => 'Parlante Bluetooth Boom Mini', 'categoria' => 'Audio',
        'precio' => 180.00, 'stock' => 15, 'destacado' => false,
        'imagen' => 'https://picsum.photos/seed/boommini/600/400',
        'descripcion' => 'Resistente al agua IPX7, 12 h de reproducción.'],
    ];
}

```

```
[ 'id' => 6, 'nombre' => 'Cámara Web Clara 1080', 'categoria' => 'Accesorios',
  'precio' => 130.00, 'stock' => 0, 'destacado' => false,
  'imagen' => 'https://picsum.photos/seed/clara1080/600/400',
  'descripcion' => 'Full HD con enfoque automático y micrófono dual.' ],
];
}
```

DEFINICIÓN · DATOS ESTÁTICOS

Datos estáticos: son datos escritos directamente en el código (aquí, arreglos PHP), en lugar de provenir de una base de datos. Sirven para practicar la capa de vistas sin la complejidad de la persistencia.

Formas de enviar datos del controlador a la vista (cualquiera es válida):

- `view('vista', ['clave' => $valor])` — arreglo asociativo (la forma usada arriba).
- `view('vista', compact('productos'))` — atajo cuando la variable se llama igual que la clave.
- `view('vista')->with('productos', $productos)` — encadenado.

En la vista, cada clave se convierte en una variable disponible: `$tienda`, `$categorias` y `$productos`.

06 Cómo la vista recibe los datos

La vista recibe las variables que el controlador le pasó. El siguiente esqueleto muestra únicamente el puente de datos: cómo recorrer los productos. No es la solución — el diseño de la interfaz es su trabajo.

```
resources/views/tienda/index.blade.php (esqueleto mínimo) Copiar

{{-- Solo muestra el puente de datos. El diseño lo construye usted. --}}
<h1>{{ $tienda['nombre'] }}</h1>

@forelse ($productos as $producto)
    {{-- Aquí irá su tarjeta de producto maquetada con Tailwind --}}
    <p>{{ $producto['nombre'] }} – Bs {{ number_format($producto['precio'], 2) }}</p>
@empty
    <p>No hay productos disponibles.</p>
@endforelse
```

ADVERTENCIA

Este esqueleto es solo el punto de partida del puente de datos. Entregar la vista así, sin maquetación con Tailwind ni la estructura requerida en la sección 07, no cumple con la práctica.

07 Trabajo a desarrollar: las vistas

Construya las vistas de la aplicación con Blade y Tailwind CSS v4. Como mínimo, su proyecto debe respetar la siguiente estructura de archivos:

```
estructura de vistas Copiar

resources/views/
├── layouts/
│   └── app.blade.php      ← layout base (nav + footer + slot/yield)
├── components/
│   └── producto-card.blade.php ← componente de tarjeta (sugerido)
└── tienda/
```

└─ index.blade.php
 └─ show.blade.php

← catálogo (OBLIGATORIO)
 ← detalle del producto (BONUS)

El catálogo (index.blade.php) debe contener, como mínimo, los siguientes bloques:

BLOQUE	REQUISITO MÍNIMO
Barra de navegación	Nombre de la tienda, enlaces de categorías y un ícono de carrito con contador (estático). Debe adaptarse en móvil.
Encabezado / hero	Nombre de la tienda, su lema y una llamada a la acción.
Categorías	Chips o botones por categoría, recorriendo \$categorias (visuales, sin lógica de filtrado).
Grid de productos	Recorre \$productos con @forelse; cada tarjeta muestra imagen, nombre, categoría, precio y un botón "Agregar". Grid responsivo: 1 columna en móvil, 2-3 en escritorio.
Estado del producto	Distinga visualmente los productos destacados (destacado = true) y los agotados (stock = 0) con insignias / badges.
Pie de página	Información de la tienda, enlaces y créditos.

Requisitos técnicos de Tailwind y Blade:

- Diseño mobile-first y responsivo (prefijos `sm:`, `md:`, `lg:`).
- Uso de Flexbox y Grid mediante utilidades de Tailwind.
- Estados interactivos con `hover:` y `focus:` (botones y tarjetas).
- Un layout base reutilizado por las vistas (herencia `@extends/@yield` o componente `<x-app-layout>`).
- Al menos un componente Blade (por ejemplo, la tarjeta de producto).

08 Secuencia de commits sugerida

Se valora el historial, no solo el resultado final. Trabaje con commits atómicos:

#	MENSAJE DE COMMIT	CONTENIDO
1	chore: rutas y controlador	web.php + TiendaController con datos estáticos
2	feat: layout base	layouts/app.blade.php (nav y footer)
3	feat: hero del catálogo	Encabezado con nombre y lema
4	feat: grid de productos	Recorrido de \$productos + tarjeta
5	feat: badges de estado	Destacado y agotado
6	feat: barra de categorías	Chips desde \$categorias
7	style: responsividad y hover	Ajustes mobile-first y estados
8	docs: README	Documentación del repositorio

09 Entregables

- Repositorio en GitHub llamado INF560-P01-Apellido (reemplace Apellido por el suyo).

- Historial con commits atómicos (mínimo 8).
- Un archivo README.md con: nombre, descripción, instrucciones de ejecución (`npm run dev` y `php artisan serve`).
- El enlace del repositorio, entregado mediante `virtual.uatf.edu.bo` en el apartado Práctica 1.

ADVERTENCIA

No suba las carpetas `vendor/` ni `node_modules/` al repositorio: ya están excluidas en el `.gitignore` que Laravel trae por defecto.

10 Rúbrica de evaluación

CRITERIO	PTS
Flujo ruta, controlador, vista: los datos del controlador llegan y se muestran	20
Estructura de vistas: layout base reutilizado + al menos un componente Blade	15
Catálogo: grid recorriendo <code>\$productos</code> con <code>@forelse</code>	20
Tailwind: responsividad mobile-first, Flexbox/Grid, estados hover/focus	20
Badges de estado (destacado / agotado) y barra de categorías	10
Commits atómicos y con mensajes claros (mínimo 6)	10
README completo con capturas	5
Total	100
<i>Bonus: página de detalle de producto (ruta <code>tienda.show</code>) funcionando</i>	+10

11 Bonus — Detalle de producto (opcional)

Implemente la vista `resources/views/tienda/show.blade.php` para mostrar un producto individual. La ruta y el método `show` ya están provistos: reciben el `id`, buscan el producto en el arreglo y lo envían a la vista.

Enlace cada tarjeta del catálogo al detalle usando el helper de ruta con nombre:

en su tarjeta de producto (Blade)

Copiar

```
<a href="{{ route('tienda.show', $producto['id']) }}">
  Ver detalle
</a>
```